Borrowed Fourier compiler proof reading copy

Corrected review annotation and preserved submitted proof

Publication review dated 12 October 2026 UTC

Status of this document. This is an editorial reading copy, not the original submitted PDF. The following pages retain the submitted mathematical construction, lemmas, proofs, equations, finite-check record and references. Its source manuscript is dated 13 October 2026. The original is preserved separately. One nonmathematical workflow paragraph has been omitted, and one sentence describing the origin of the authored tests has been restated without workflow context. No construction, proof or mathematical formula has been removed or silently corrected.

Corrected comparison. The construction is a mathematically valid alternative one-bank implementation of an already available exact-width $O(1 + \log^2 r)$ Fourier layer theorem. The submitted comparison against only OpenAI's written $O(\log^4 r)$ bound is incomplete. OpenAI's formal source, introduced on 8 October 2026, already gives the exact-width, arbitrary-input quadratic-logarithmic compiler, charged polynomial preparation and a two-bank route. The original comparison paragraphs below are retained as the submitted record and are superseded by this annotation. No new local asymptotic exponent or global asymptotic improvement over that complete prior framework is established. Historical originality or a total-constant advantage of the one-bank regularizer remains unestablished.

Exact prior. At revision `fd4aeeb2ee4fc729c18d98444fed42fd0529eeeb`, `FourierWords.lean` supplies `sound_ballad`, `creates_ballad`, and the bound $20{,}000{,}000(e + 1)^2 + 7$ for $r \leq 2^e$. `LocalCompiler.publish` supplies its preparation interface. The introducing commit is the 8 October 2026 formalization commit.

Exposition qualifications. In Section 2, $L(v)$ has shape $a \times e$ and $U(w)$ has shape $e \times e$. Assemble parallel children in generalized disjoint layers and split mixed types once globally, adding at most a factor two overall. For conjugate-closed scalar preparation, cache a value/conjugate pair at each arithmetic DAG node, evaluating specified roots at their inverses. This preserves polynomial cost. Exact width concerns the emitted data-coordinate word, not the evaluator's total memory or its stored schedule.

Execution boundary. The submitted $455 + 10$ positive cases and six negative controls are author-recorded results; the authored verifier was not rerun. A separate fresh implementation passed bounded exact checks, summarized in the accompanying audit and result files. Those synchronization checks used conventional C-tensor butterflies. No imported fast tensor engine, all-length CRT/Bluestein implementation or Lean build was executed. The global sub-$N \log N$ bound is a mathematical implication using existing charged theorem dependencies, not an executed end-to-end performance result. This document and the result metadata alone are not a self-contained reproducibility package.

Reading order. Read the corrected audit first. Read the following submitted proof for the full algebra and construction. Its original test counts and historical comparison must be interpreted under the qualifications above.

A Fourier Compiler with Borrowed Registers

An $O(\log^2 r)$ local construction and its tensor transfer

Research candidate, with exact checks — 13 October 2026

What this attempt produces. An explicit compiler for the full r-point Fourier matrix on exactly r data coordinates, using $O(1+\log^2 r)$ layers of disjoint shears and invertible monomial maps. Its temporary data are borrowed from existing coordinates and restored exactly. Coefficients and schedules have polynomial preparation cost. It works on arbitrary inputs and at every positive local length r, including primes.

The inspected OpenAI compiler uses $O(1+\log^4 r)$ layers. Replacing its gate-by-gate workspace simulation with the construction below changes this local bound. **This is a candidate compiler improvement, not a confirmed historical-first theorem.** General reversible accumulation, Fourier factorization, and tensor power savings are prior work.

The derived all-input bound. If an existing, fully charged tensor engine computes the fixed kernel's kth tensor power in $O(2^k(k+1)^{1-a})$ work for a fixed $0<a<1$, the new compiler gives

$$\boxed{T(N)=O\Big(N(\log N)^{1-a}(\log\log N)^{1+a}\Big)=o(N\log N).}$$

The old compiler in the same transfer gives the exponent $3+a$ on $\log\log N$. The candidate therefore improves that *displayed upper-bound expression* by $(\log\log N)^2$, for the same tensor engine. It does not improve the engine's exponent a, nor establish a matching lower bound for the old algorithm.

Claim	Evidence and boundary
Local algebra and layer bound	Constructive proofs in this note; executable local compiler.
Exact finite checks	455 positive modular cases, 10 characteristic-zero cases, 6 rejection/corruption controls.
Full sub-$N\log N$ transfer	A proved implication using an existing tensor engine and the published all-length reduction; the global engine was not executed here.
Publication novelty	Unconfirmed. Related in-place accumulation literature needs a complete comparison.

Model. Exact complex arithmetic; unrestricted scalar magnitudes; a specified supplied root of unity; unit-cost operations on logarithmic-size indices. Scalar preparation and array movement are charged. This is an asymptotic arithmetic result, not a floating-point, bit-complexity, or measured CPU speedup. The local prototype is deliberately much slower than an ordinary FFT at the tested sizes.

1. A convolution update with one arbitrary scratch bank

We count layers of shears $x_i \leftarrow x_i + cx_j$ on disjoint pairs, and invertible monomial layers. The latter are permutations and nonzero coordinate scalings. Scalar coefficients are prepared separately. No data coordinate is assumed zero.

Let T be an $a \times e$ Toeplitz matrix, with $T_{ij} = h_{i-j}$. Choose a power of two $q \geq a + e - 1$. Embed the signed coefficients into a length-q circular-convolution matrix K: the coefficient of residue d is h_d for $-(e-1) \leq d < a$ and zero elsewhere. Let $E : \mathbb{C}^e \to \mathbb{C}^q$ inject into the first e positions and let $R : \mathbb{C}^q \to \mathbb{C}^a$ select the first a. Distinct represented differences do not collide, and $T = RKE$.

If F_q uses the positive-exponent convention and $\lambda = F_q h^{\mathrm{circ}}$, then

$$K = F_q^{-1} \operatorname{diag}(\lambda_j) F_q.$$

Some λ_j may vanish. Define, on prepared scalars only,

$$\alpha = 1 + \sum_j \lambda_j \overline{\lambda_j}, \qquad A = K + \alpha I = F_q^{-1} \operatorname{diag}(\lambda_j + \alpha) F_q.$$

For every j, $\alpha \geq 1 + |\lambda_j|^2 > |\lambda_j|$. Thus $\lambda_j + \alpha \neq 0$ and both A and A^{-1} have in-place $O(\log(q+1))$-layer implementations by ordinary radix-two FFTs and nonzero diagonal scalings.

Lemma 1 (Arbitrary-scratch Toeplitz accumulation)**.** *On disjoint data $x \in \mathbb{C}^e$, $y \in \mathbb{C}^a$, and arbitrary $z \in \mathbb{C}^q$, the following chronological sequence computes $(x, y, z) \mapsto (x, y + Tx, z)$:*

1	$z \leftarrow z + Ex$;	2	$z \leftarrow Az$;
3	$y \leftarrow y + Rz$;	4	$z \leftarrow A^{-1}z$;
5	$z \leftarrow z - Ex$;	6	$z \leftarrow Az$;
7	$y \leftarrow y - Rz$;	8	$z \leftarrow A^{-1}z$;
9	$y \leftarrow y - \alpha REx$.		

It needs $O(\log(q+1))$ layers and $e + a + q$ data coordinates.

Proof. Writing the original scratch as z_0, the two additions to y are $RA(z_0 + Ex)$ and $-RAz_0$. Their difference is $RAEx$. The final update subtracts αREx, leaving $RKEx = Tx$. Steps 4–5 and 8 return scratch to z_0; the source is unchanged. Each injection, selection, or correction is one disjoint shear layer. □

Uniform scalar preparation. For Fourier-derived Toeplitz coefficients, every scalar is a rational expression in specified roots and rational numbers. Its conjugate is obtained by evaluating the same shared arithmetic program with those roots inverted. No input data are conjugated, and no complex zero test is used. A statement for arbitrary supplied Toeplitz coefficients requires their conjugates too, or a nonuniform choice of the nonzero shift; the uniform Fourier theorem has no such missing input.

2. Product updates and a rank-three displacement

Lemma 2 (Restored intermediate register)**.** *Suppose procedures implement* $\mathcal{A} : (x, z) \mapsto (x, z + Ux)$ *and* $\mathcal{B} : (z, y) \mapsto (z, y + Vz)$*, each restoring all other borrowed data. Then the chronological word*

$$\mathcal{A}, \quad \mathcal{B}, \quad \mathcal{A}^{-1}, \quad \mathcal{B}^{-1}$$

computes $y \leftarrow y + VUx$*, preserving both* x *and the arbitrary initial* z*.*

Proof. The target first receives $V(z_0 + Ux)$ and later loses Vz_0. The intermediate returns to z_0 before the subtraction. The two procedures may reuse the same scratch bank because each restores it completely. □

We reuse the published OpenAI compiler's Toeplitz decomposition and displacement formulas. Let $T_v = (h_{i-j})_{0 \le j \le i < v}$, with $h_0 \neq 0$, and define the reciprocal-series coefficients by

$$g_0 = h_0^{-1}, \qquad g_k = -h_0^{-1} \sum_{j=1}^{k} h_j g_{k-j}.$$

For $p = \lfloor v/2 \rfloor$, split

$$T_v = \begin{pmatrix} T_p & 0 \\ B & T_{v-p} \end{pmatrix} = \begin{pmatrix} I & 0 \\ M & I \end{pmatrix} \operatorname{diag}(T_p, T_{v-p}), \qquad M = BT_p^{-1}.$$

The cross update entries, in original indices, are

$$M_{ij} = \sum_{\nu=j}^{p-1} h_{i-\nu} g_{\nu-j}, \qquad p \le i < v, \quad j < p.$$

For interior positions of any rectangular chunk, cancellation gives

$$M_{ij} - M_{i-1,j-1} = -h_{i-p} g_{p-j}.$$

For an $a \times e$ chunk Q, let J_a, J_e be the lower shifts. Its displacement

$$D = Q - J_a Q J_e^{\mathsf{T}}$$

is the sum of three explicit outer products. Extend the interior product to the entire chunk, subtract it from D, and call the result B_0. It is zero except in its first row or column. With first unit vectors e_0,

$$D = uv^{\mathsf{T}} + e_0 (B_0)_{0,*} + \big((B_0)_{*,0} - (B_0)_{00} e_0\big) e_0^{\mathsf{T}}.$$

This is an explicit identity, with no rank discovery step. Nilpotence of the shifts yields

$$Q = \sum_{t \ge 0} J_a^t D (J_e^{\mathsf{T}})^t.$$

Each displacement term vw^{T} therefore contributes $L(v)U(w)$, where

$$L(v)_{it} = \begin{cases} v_{i-t} & i \ge t, \\ 0 & i < t, \end{cases} \qquad U(w)_{tj} = \begin{cases} w_{j-t} & j \ge t, \\ 0 & j < t. \end{cases}$$

Both are rectangular or triangular Toeplitz maps. Sections 1–2 implement their product update using a restored intermediate bank. The rank-three displacement identity is classical structured linear algebra; the proposed change is how its updates fit into the original width with small depth.

3. Constant-size partitioning proves the depth bound

At a recursive node of width $v \geq 16$, choose $b = \lfloor v/8 \rfloor$ and split each half into consecutive chunks of size at most b. For one target/source pair, write its sizes as $a, e \leq b$ and choose

$$q = 2^{\lceil \log_2 \max(2e-1, a+e-1) \rceil}.$$

This single scratch length supports both Toeplitz maps in every product term. Its total data requirement is

$$\underbrace{e}_{\text{source}} + \underbrace{a}_{\text{target}} + \underbrace{e}_{\text{intermediate}} + \underbrace{q}_{\text{FFT scratch}} \leq 3b + 4b \leq v.$$

Here $q < 4b$; the loose displayed inequality is sufficient. Select the intermediate and scratch coordinates anywhere outside the source and target chunks but within the same parent block. They may currently contain transformed source data, unprocessed targets, or previously updated targets. The exact identities restore every one of them.

Both halves have at most six chunks when $v \geq 16$; a weaker constant bound would also suffice. Thus there are at most 36 source/target pairs. Process these pairs serially, with three product updates per pair. Each product consists of four Toeplitz accumulations, each of depth $O(1 + \log v)$. The cross update therefore has depth $O(1 + \log v)$.

The two child transforms act on disjoint halves and run in parallel *as layers*. This is a description of a scalar circuit, not a claim that processor parallelism reduces sequential work. The layer bound satisfies

$$d(v) \leq \max\{d(\lfloor v/2 \rfloor), d(\lceil v/2 \rceil)\} + O(1 + \log v).$$

For $v < 16$, direct row updates in decreasing order use bounded depth and exactly the existing coordinates. Summing along a balanced recursion path proves

$$\boxed{d(r) = O(1 + \log^2 r).}$$

Explicit layer accounting. The emitted radix-two word has at most $1 + 3\log_2 q$ layers. An invertible circulant word and its inverse each have $3 + 6\log_2 q$ layers. A Toeplitz accumulation uses at most $17 + 24\log_2 q$ layers. Three nested product updates cost at most $204 + 288\log_2 q$ layers per chunk pair. These intentionally loose constants also cover zero coefficients and identity operations.

Why there is no hidden workspace. Every emitted instruction addresses only the original r data coordinates. A layer validator checks disjointness, coordinate range, and nonzero diagonal scalings. An inverse word reverses layers, negates shear coefficients, and reciprocates scaling coefficients. No clean data allocation or destructive clearing is part of the compiled word. Prepared scalar programs, schedule storage, and indices are separate and polynomial in r.

Preparation. The reciprocal recurrence, cross entries, explicit outer products, small FFT coefficients, conjugates, and inverses are computable in polynomial scalar/index work. Direct evaluation of the displayed finite sums suffices. Stored scalar expressions remain shared programs, not expanded expression trees. The largest temporary FFT length is less than $4b \leq r/2$; the previous allowance of power-of-two roots of order below $8r$ is more than sufficient.

4. From Toeplitz layers to the full Fourier matrix

For a primitive rth root u, define

$$H_0 = 1, \quad H_j = \prod_{s=1}^{j}(1-u^s), \qquad p_j = (-1)^j u^{j(j-1)/2}, \qquad T_{ij} = \frac{[i \geq j]}{H_{i-j}}.$$

All H_j, $j < r$, are nonzero. The classical Newton/Vandermonde factorization is

$$\boxed{F_r(u) = D_H\, T\, \mathrm{diag}(p_j/H_j)\, T^{\mathsf{T}}\, D_H,} \qquad D_H = \mathrm{diag}(H_j). \tag{1}$$

For completeness, let column j of a unit upper triangular matrix P contain the coefficients of $\prod_{s<j}(z-u^s)$. Then $F_r P = N$ is lower triangular and

$$N_{ij} = p_j H_i / H_{i-j} \quad (i \geq j), \qquad N = D_H T\, \mathrm{diag}(p_j).$$

Uniqueness of the unit-lower/diagonal/unit-upper factorization, together with $F_r = F_r^{\mathsf{T}}$, gives $F_r = N\, \mathrm{diag}(N_{jj})^{-1} N^{\mathsf{T}}$. Since $N_{jj} = p_j H_j$, this is (1). Width one is immediate.

Theorem 3 (Local compiler)**.** *For every $r \geq 1$, (1) has an explicitly prepared word on exactly r data coordinates in $O(1+\log^2 r)$ layers of disjoint shears and invertible monomials. It computes the full Fourier transform for every complex input. Preparation is polynomial in r and uses rational operations on u, suitable power-of-two roots, and their inverses.*

Proof. Apply Section 3 to T. Reversing the order of layers and transposing each instruction gives T^{T} at the same depth; a shear's transpose exchanges its source and target. The three diagonal matrices in (1) are invertible monomial layers. Mixed layers on disjoint regions split into a monomial and a shear layer at constant overhead. The preparation and denominator guarantees were established above. □

Fixed-kernel synchronization. The published tensor engine uses

$$C = \tfrac{1}{2}\begin{pmatrix} 1+i & 1-i \\ 1-i & 1+i \end{pmatrix}.$$

The known constant-size conversion from an arbitrary shear to monomials and C still applies. With $H' = \left(\begin{smallmatrix} 1 & 1 \\ 1 & -1 \end{smallmatrix}\right)$,

$$H'\, \mathrm{diag}(2,1) H'\, \mathrm{diag}(-3,1) H' = \begin{pmatrix} -8 & -10 \\ 0 & -6 \end{pmatrix}.$$

Diagonal conjugation converts the resulting nonzero upper shear coefficient $5/4$ into any $t \neq 0$, and H' itself is a diagonal sandwich of C. To handle any μ, including zero, split it into $t_1 = 1 + |\mu|^2$ and $t_2 = \mu - t_1$; both are nonzero. Thus six C slots and a constant number of monomial slots suffice per shear layer. Conjugates concern prepared scalars only. Padding with singleton identities creates a common slot pattern at the same asymptotic depth. This conversion is credited to the existing framework, not claimed as new here.

5. The all-length algorithm and full cost

Imported premise. Fix an existing exact algorithm for $C^{\otimes k}$ with work $O(2^k(k+1)^\theta)$, where $0 < \theta = 1 - a < 1$. Its guarantee must include arbitrary values in every auxiliary role, scalar preparation, indexing, initialization and data movement. The finite tensor construction is the source of $a > 0$; this note does not discover it.

For pairwise coprime small axes $r_1, \dots, r_\ell$ and $R = \prod r_j$, compile each axis by Section 4. A tensor product of monomial slots costs $O(R)$ using accumulated mixed-radix addresses and scalar products. A simultaneous C slot partitions the array into sectors of sizes 2^k. Each sector carries exactly $C^{\otimes k}$, with $k \le \ell$, and sector sizes sum to R. Prefix-tree traversal performs packing, unpacking and sector discovery in $O(R)$ work. Consequently the compiled tensor transform costs

$$O(R(\ell+1)^\theta(1+\log r_{\max})^2) + \mathrm{poly}(\ell, r_{\max}, \log(R+2)). \tag{2}$$

This uses the same array adapters as the published reduction, with a different local slot count.

Complete algorithm, at the mathematical subroutine level.

1. Select the largest initial odd-prime product $P_\ell \le 2N$, then the least $e \ge 0$ with $L = 2^e P_\ell \ge 2N$. Thus $2N \le L < 4N$, $r_{\max}$ is polynomial in $\log N$, and $\ell = \Theta(\log N / \log\log N)$.

2. Prepare roots and all local compiler words. CRT indexing converts F_L to a tensor product over the odd-prime axes and the one power-of-two axis. Use (2) for the former and an ordinary short FFT for the latter.

3. Apply Bluestein's identity with $\eta = \exp(\pi i/N)$: $\eta^{2jk} = \eta^{j^2}\eta^{-(k-j)^2}\eta^{k^2}$. Set $f_j = \eta^{j^2}x_j$ for $j < N$ and zero otherwise; set $g_{d \bmod L} = \eta^{-d^2}$ for $|d| < N$, and zero elsewhere. Transform both, multiply pointwise, apply the inverse working transform, and return $\eta^{k^2}(f * g)_k$, $k < N$. The fixed kernel's transform is charged too.

Generate chirps in $O(N)$ work: their successive ratios are η^{2j+1}, updated by multiplication by η^2 (invert for negative phases). There are three working transforms, $O(N)$ other array operations, and polynomial-in-$\log N$ local preparation. A single specified supplied root of order $D_* = (2N)L\,2^{\lceil \log_2(16L)\rceil} < 1024N^3$ supplies all roots by charged powering. Scalar data have unrestricted size; addresses and integer quantities are polynomially bounded in N. These are the published model's conventions, not a promise that an ordinary supplied Nth root alone suffices.

Substitution into (2), with finite small lengths handled directly, gives

$$T(N) = O(N(\log N)^\theta(\log\log N)^{2-\theta}).$$

For fixed $a = 1 - \theta > 0$, divide by $N\log N$ and put $t = \log\log N$:

$$\frac{T(N)}{N\log N} = O(e^{-at}t^{1+a}) \longrightarrow 0.$$

Thus the implication really is sub-$N\log N$ on arbitrary inputs. The executable files implement and check the local compiler only; they do not implement the enormous imported tensor engine or benchmark the composed all-length algorithm.

6. Verification, novelty, and research provenance

Editorial annotation. The counts below are the authored suite's recorded results, not a rerun. The novelty discussion below is retained as submitted; the existing formal quadratic-depth compiler and two-bank construction supersede its incomplete comparison. See the opening correction and accompanying audit.

Exact check	Cases
Toeplitz accumulations, arbitrary nonzero scratch	108
Nested product accumulations, arbitrary intermediate/scratch	25
Full Fourier vectors over verified prime fields	54
Complete basis-column comparisons over those fields	232
Full triangular Toeplitz vectors, plus inverse round trips	36
Characteristic-zero scratch, Toeplitz, and Fourier checks	10
Malformed programs and deliberate corruption controls	6

Modular Fourier lengths were $1, 2, 3, 4, 5, 7, 8, 11, 15, 16, 17, 23, 24, 31, 32, 33, 48, 64$. Each Fourier comparison uses the literal quadratic sum, not another call to the compiler. Complete bases are checked for each listed length at most 33. The complex checks use exact rational coordinates in $\mathbb{Q}(\zeta_{16})$ or $\mathbb{Q}(i)$, including Fourier width 16 and the first recursive Toeplitz width. They use the actual complex regularizer $1 + \sum |\lambda|^2$.

The modular backend instead picks a scalar avoiding the finite eigenvalue set. This checks the algebraic identities but is not the complex proof's preparation method. The characteristic-zero suite and the nonzero-denominator proof address that difference. No floating-point tolerance is used. The authored checks reported here are not external independent review or Lean formalization. [Editorial restatement of test provenance.]

No practical speed claim. At width 64 the literal prototype emits 102,603 layers and 1,273,024 scalar arithmetic operations. Its role is to validate a width-preserving compiler, not to outperform a conventional 64-point FFT. The positive asymptotic bound needs the separate tensor engine.

Novelty boundary. Independent derivation does not establish priority. Dumas–Grenet's extended paper was located and Sections 6.1–6.4 inspected: it already gives in-place circulant/Toeplitz accumulation and displacement-based triangular products. Its Algorithms 57 and 59 order child calls around a cross update; Algorithm 61 uses triangular multiplication and solving. OpenAI already supplies parallel children, BT_p^{-1}, and the rank-three chunk formulas. Our candidate change to that scaffold is the regularized one-bank convolution update and product commutator: $O(b)$ borrowed coordinates replace $O(b \log b)$ gate storage, allowing constant-fraction chunks. Whether earlier methods imply this same depth remains open in this audit. The tensor engine and all-length reduction also remain prior work.

Primary comparisons.

1. OpenAI, *An explicit power saving for the exact discrete Fourier transform*, 25 September 2026. Sections `network.tex`, `local.tex`, `synchronization.tex`, `all-lengths.tex`. Repository source.
2. Ryan Shea, *Finite Network Improvements for the Exact Discrete Fourier Transform.* Transfer framework and provenance.
3. Jacob Sussman and cited community contributors, Fourier/tensor proof repository. The inspected README reports a 10 October 2026 Fourier saving $8762478/10^{10}$; no new saving parameter is claimed here.
4. J.-G. Dumas and B. Grenet, *In-place accumulation of fast multiplication formulae*, ISSAC 2024, arXiv:2307.12712v3. Extended paper: *Fast in-place accumulation*, arXiv:2302.13600v7. DOI: 10.1016/j.jsc.2025.102523.